FORGEAI

ENGINEERING INTELLIGENCE

PROJECT SPECIFICATION

ForgeAI

Private, evidence-backed engineering decisions through deterministic multi-agent analysis.

VERIFIED AMD RADEON CLOUD INFERENCE

ForgeAI transforms an ambiguous feature request and local project documentation into a validated Engineering Decision Package. Five bounded AI roles discover constraints, select trusted evidence, propose a solution, challenge it, and revise it inside an application-owned workflow.

FIELD	VERIFIED VALUE
Track	Track 2 - Development & Local Deployment of Private AI Agents
Verified live model	AMD Radeon Cloud - DeepSeek-V4-Flash
Repository	github.com/AbhishekHerbertSamuel/forgeai
Verified commit	261a01568508a478e875d8bca47bd89be55b82f0

DOCUMENT GUIDE

Table of contents

A capability-oriented specification for Track 2 - Development & Local Deployment of Private AI Agents.

Section	Title
01	Executive Summary
02	Problem and Opportunity
03	Track 2 Fit
04	System Architecture
05	Workflow Orchestration
06	Agent Design
07	Evidence and Provenance
08	Persistence and API
09	AMD Integration
10	Inference Optimization
11	Live Validation
12	Product Experience
13	Decision Artifact
14	Engineering Quality
15	Local Radeon / ROCm Deployment Plan
16	Reproducibility
17	Limitations and Roadmap
18	Validation Scope and Claims Boundary
19	Participant Contribution
20	Conclusion
21	Source Basis

STATUS LANGUAGE

Label	Meaning
Implemented	Present in the pinned repository and covered by offline verification.
Verified	Observed in the persisted full AMD Radeon Cloud workflow.
Planned	Architectural deployment path only; not completed or benchmarked in the submitted MVP.

01 / EXECUTIVE SUMMARY

Engineering decisions, not AI theatre

Software feature requests rarely arrive with complete constraints, reliable evidence, or an audit trail. ForgeAI creates that missing decision layer. It searches local engineering documents, asks targeted clarification questions, and drives five specialized reasoning stages through a fixed, inspectable state machine.

> **The differentiator is where trust is placed. Models generate bounded judgments; deterministic software owns identifiers, provenance, state transitions, validation, persistence, and the final artifact.**

5	7	0	0
bounded reasoning roles	trusted evidence items in verified run	provider retries in verified run	output repairs in verified run

Product outcome

- A structured Engineering Decision Package rather than an ephemeral chat transcript.
- Every material conclusion is tied to evidence, labeled inference, or unresolved uncertainty.
- Critique is explicit, independent, and reconciled finding by finding.
- The workflow can pause for human clarification and resume without rerunning Discovery.
- Provider portability is preserved through a small OpenAI-compatible interface.

Submission evidence

Item	Status	Evidence
Working product	Implemented	FastAPI, CLI, React dashboard, SQLite and deterministic workflow
AMD inference	Live verified	Completed DeepSeek-V4-Flash workflow via Radeon shared API
Quality	Offline verified	156 backend tests, 16 frontend tests, 93% backend coverage
Public repository	Ready	github.com/AbhishekHerbertSamuel/forgeai
Dedicated ROCm deployment	Future	Architecture-compatible but not implemented or benchmarked

02 / PROBLEM AND OPPORTUNITY

From feature request to defensible decision

ForgeAI targets the gap between requirement intake and implementation: the moment when teams must understand existing decisions, identify constraints, compare options, and document why an approach is safe.

> **Engineering review delays, missed constraints, and undocumented assumptions create rework, security exposure, and slower delivery - especially in regulated systems such as payments.**

Why current workflows fail

Failure mode	Engineering consequence	ForgeAI response
Ambiguous request	Teams implement different interpretations	Discovery normalizes goals and pauses for ordered clarification
Scattered local knowledge	ADRs and policies are overlooked	Deterministic indexing and lexical retrieval surface cited passages
Opaque AI output	Recommendations cannot be audited	Strict schemas, evidence status and persisted stage outputs
Unchallenged proposal	Security and operational risks survive review	Independent Critic followed by explicit Revision dispositions
Provider lock-in	Inference choices reshape application logic	Provider-neutral OpenAI-compatible port

Primary users

- Software engineers and technical leads evaluating implementation options.
- Engineering managers coordinating decisions across teams.
- Architects, platform engineers, security reviewers, and privacy partners.
- Teams operating with confidential local documentation or offline-capable infrastructure.

Representative scenarios

- Architecture and feature feasibility review.
- Security, compliance, API, database, and backward-compatibility analysis.
- Change-impact assessment against existing ADRs, schemas, and policies.
- Persistent decision history for later engineering review.

03 / TRACK 2 FIT

Track 2 - Development & Local Deployment of Private AI Agents

Track 2 capability	ForgeAI implementation	Status
Local knowledge retrieval / RAG	Local document ingestion, SQLite FTS5 retrieval, stable chunks and exact citations	Implemented
Multi-step task planning	Discovery -> Evidence -> Solution -> Critique -> Revision	Implemented
Persisted workflow memory	Durable state, events, clarification answers, stage outputs and EDPs	Implemented
Privacy protection	Local corpus, sanitized diagnostics and no model-controlled commands or repository mutation	Implemented
Tool invocation	Application-controlled retrieval and persistence adapters	Implemented with deterministic control
Structured output	Strict Pydantic contracts, safe JSON normalization and one bounded repair	Implemented
Human-in-the-loop	Ordered clarification pause and same-workflow resume	Implemented
AMD inference	Full five-agent DeepSeek workflow through AMD Radeon Cloud shared Model API	Verified
Dedicated Radeon / ROCm deployment	OpenAI-compatible substitution path prepared	Planned

Product boundary

ForgeAI is not an autonomous coding agent. Product agents cannot execute commands, mutate repositories, choose tools, or bypass validation. This narrower boundary is intentional: engineering judgment remains inspectable and human-accountable.

04 / SYSTEM ARCHITECTURE

One engine, two transports, explicit boundaries

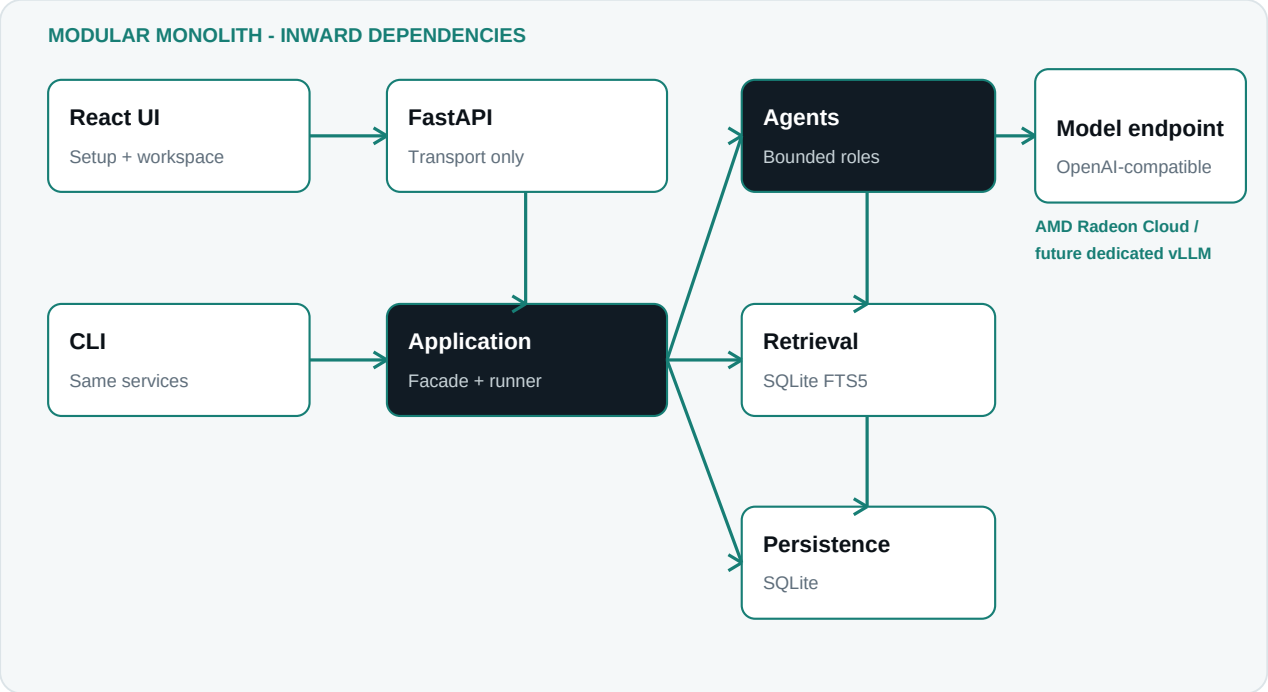


Figure 1. Modular-monolith architecture. Both CLI and REST invoke the same application services; transport code contains no workflow logic.

Architecture principles

- Domain concepts and invariants remain provider-independent.
- Application services own use cases, transitions, retries, validation and orchestration.
- Adapters implement SQLite, retrieval, document parsing, clocks, IDs and model inference.
- FastAPI maps HTTP contracts to application commands without direct agent, provider or database access.
- The React application consumes the public API and never relies on database internals.

Deployment posture

The current product is a local modular monolith: one backend process, one SQLite database, and a separately built React frontend. This is the smallest topology consistent with privacy, auditability, and portability.

05 / WORKFLOW ORCHESTRATION

Deterministic control around probabilistic models

FIXED, BOUNDED STATE MACHINE

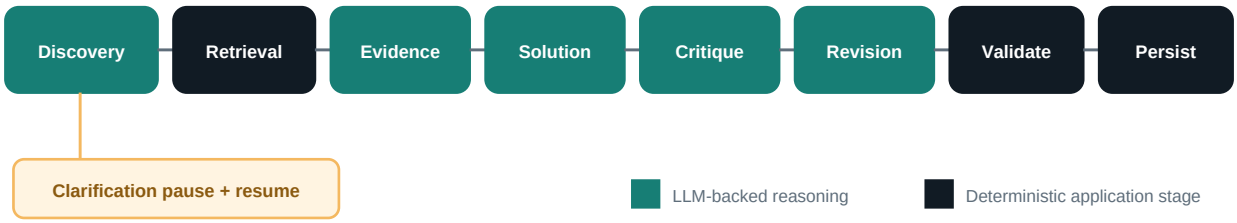


Figure 2. Fixed state machine. Discovery may pause for clarification; resume continues at Retrieval and never reruns Discovery.

Workflow invariants

- Every stage has typed, versioned input and output contracts.
- Stage ordering and retry policy live in application code, never only in prompts.
- Invalid or partial model output cannot be converted into apparent success.
- Events, timings, stage output, provider metadata and terminal state are persisted.
- Cancellation and failures are explicit terminal outcomes.
- Validation checks evidence provenance, known references, exact dispositions and stage order before persistence.

Stage	Responsibility	Deterministic enforcement
Discovery	Normalize request and identify unknowns	Clarification state and ordered questions
Retrieval	Search local evidence	FTS ranking, deduplication and stable order
Evidence	Select and assess candidates	Trusted provenance reconstruction
Solution	Alternatives, recommendation, claims and risks	Evidence-label reconstruction and epistemic rules
Critique	Find unsupported or unsafe reasoning	Category mapping, IDs and evidence references
Revision	Update solution and resolve every finding	Sparse patch, exactly-once disposition and label validation

06 / AGENT DESIGN

Small model-facing contracts, stable public contracts

Agent	Model generates	ForgeAI reconstructs or controls
Discovery	Goal, actors, constraints, assumptions, unknowns, queries and questions	Schema version, pause/resume behavior and persistence
Evidence	Candidate labels and relevance explanations; string conflicts and gaps	All IDs, paths, lines, excerpt, hash, score and order
Solution	Alternatives, trade-offs, risks, claims and guidance using short evidence labels	Public evidence IDs, schema version and zero-evidence rules
Critique	Findings using short category and evidence labels	Public enums, stable finding IDs and trusted references
Revision	Sparse solution patch and finding-label dispositions	Full revised package, public IDs and exactly-once coverage

Concrete selection	Trusted reconstruction
Model selects: C1, C3	ForgeAI reconstructs: document ID - source path - line range - exact excerpt - file hash - evidence ID

Structured-output discipline

- JSON only, with one compact valid example per prompt.
- Aliases, wrappers, extra keys, surrounding prose and hidden chain-of-thought are forbidden.
- Every array item type is explicit: strings are distinguished from objects.
- A provider-neutral normalizer accepts valid JSON, one harmless fence, or one complete object in harmless prose.
- Ambiguous, multiple, incomplete or schema-invalid objects fail safely.
- One repair attempt receives sanitized field paths and validation categories, never raw model output.

Why this matters

Reasoning models are strongest when asked to make bounded judgments, not reproduce deterministic metadata. ForgeAI removes that burden while keeping the public EDP and API contracts unchanged.

07 / EVIDENCE AND PROVENANCE

Trust retrieval, not copied model metadata

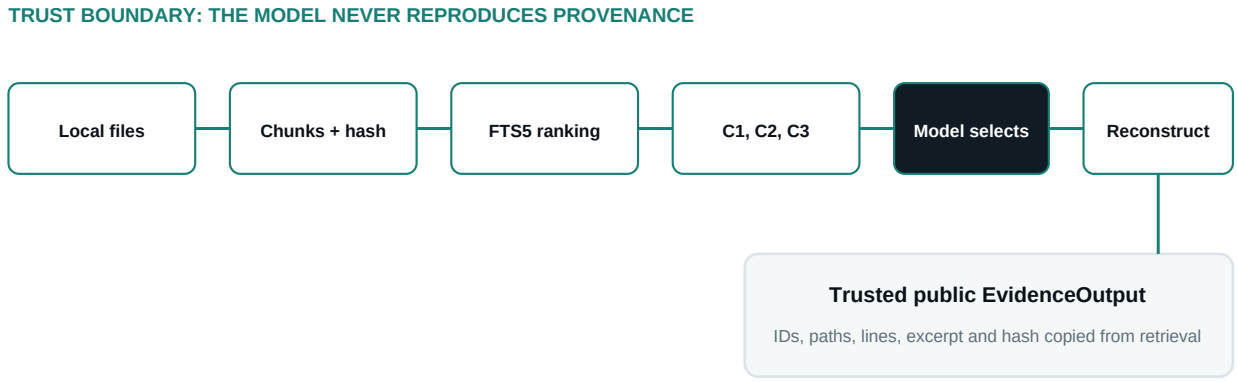


Figure 3. Evidence selection by internal label. The model chooses candidates; deterministic software reconstructs immutable provenance from the retrieval snapshot.

This prevents the model from fabricating, paraphrasing, or subtly corrupting citations while still allowing it to judge which evidence is relevant.

Document pipeline

- Recursively scans Markdown, text, JSON, YAML, Python, Java, TypeScript/TSX, JavaScript/JSX and SQL.
- Retains project ID, relative path, detected type, modification time and content hash.
- Chunks deterministically at up to 80 lines or 4,000 characters.
- Creates stable SHA-256 document and chunk identifiers.
- Detects unchanged, modified, deleted and duplicate files.
- Ranks with SQLite FTS5 and deterministic tie-breaking; no embeddings are used.

Immutable EvidenceItem fields

Trusted field	Source of truth
evidence_id, document_id, chunk_id	Deterministic retrieval candidate
source_path, start_line, end_line	Indexed source record and chunk
exact_excerpt	Exact indexed chunk text
file_hash	Indexed source-content hash
relevance_score	Deterministic lexical retrieval
relevance_explanation	Only field generated by Evidence model for a selected candidate

08 / PERSISTENCE AND API

Durable state behind a transport-only interface

SQLite persistence

- Projects, indexed files, chunks, workflows, stage outputs, events, timings, EDPs and idempotency records.
- Foreign keys, WAL mode, busy timeout, migrations and parameterized SQL.
- Optimistic concurrency protects workflow transitions.
- Canonical JSON provides durable structured artifacts.

Versioned REST surface

Method	Endpoint	Responsibility
GET	/api/v1/health/live	Process liveness
GET	/api/v1/health/ready	Application readiness
POST	/api/v1/projects/index	Index a server-local project directory
POST	/api/v1/projects/search	Search indexed project evidence
POST	/api/v1/workflows	Create and launch a workflow; optional idempotency key
GET	/api/v1/workflows/{workflow_id}	Status, stages, timings and failures
POST	/api/v1/workflows/{workflow_id}/resume	Submit ordered clarification answers
POST	/api/v1/workflows/{workflow_id}/cancel	Cancel an eligible workflow
GET	/api/v1/workflows/{workflow_id}/events	Poll ordered events
GET	/api/v1/workflows/{workflow_id}/edp	Return JSON or Markdown EDP

The generated contract is OpenAPI 3.1.0. API errors expose only a code, message, request ID and timestamp; they never include stack traces, filesystem paths or provider secrets.

09 / AMD INTEGRATION

Portable inference validated on Radeon Cloud

ForgeAI communicates through an application-owned OpenAI-compatible interface. The successful live workflow used the AMD Radeon shared model API with DeepSeek-V4-Flash in JSON-object mode.

Configuration	Verified value
Provider mode	openai_compatible
Logical model	DeepSeek-V4-Flash
Served model	radeon-deepseek/DeepSeek-V4-Flash
Response format	json_object
Timeout	300 seconds
Provider retries	0
Base URL host	developer.amd.com.cn

> **Current execution scope: The submitted MVP completed a full workflow using AMD Radeon Cloud's shared DeepSeek Model API. This demonstrates verified AMD-hosted inference. It does not claim participant-managed local ROCm or dedicated-vLLM deployment.**

Provider guarantees

- No AMD- or DeepSeek-specific branch exists in domain or orchestration code.
- Live mode fails fast when the model is missing; Qwen is not silently selected.
- Mock mode remains credential-free and deterministic.
- Telemetry records model, latency, usage, finish reason, retries and repairs without logging prompts, bodies, URLs or secrets.
- Transient HTTP and transport failures receive bounded retries; authentication and schema failures do not.

Compatibility boundary

The same application can target a future dedicated Radeon vLLM deployment by changing endpoint configuration. Dedicated ROCm serving, model quantization, GPU utilization and hardware benchmarking remain future work and are not claimed as implemented.

10 / INFERENCE OPTIMIZATION

Application-level inference optimization

ForgeAI reduces model work by assigning deterministic responsibilities to software and restricting every stage to a minimal model-facing contract.

Optimization	Engineering effect
Short labels such as C1, E1, F1 and K1	Reduces long identifier reproduction and output-token load
Trusted deterministic reconstruction	Removes paths, hashes, excerpts and IDs from model generation
Sparse Revision patches	Avoids regenerating the complete Solution
json_object response mode	Reduces formatting and parsing failures
Stage-specific output limits	Controls latency while preventing truncation
One bounded repair attempt	Caps worst-case model calls and latency
Sanitized semantic repair	Corrects contract violations without resending raw output
Provider retries set to zero in verified run	Produces a clean end-to-end measurement

Claims boundary

> **These are application-layer inference and reliability optimizations. Kernel-level ROCm tuning, quantization, GPU utilization measurement and dedicated-vLLM benchmarking were not completed and are not claimed.**

Optimization objective

- Reduce unnecessary output tokens without weakening public schemas.
- Keep provider calls bounded and failure behavior observable.
- Prevent reasoning models from reproducing values deterministic software already knows.
- Retain a portable baseline across OpenAI-compatible model servers.

11 / LIVE VALIDATION

One complete five-agent workflow, measured end to end

106.29 s	106.22 s	20,556	10
total command latency	persisted measured stage time	total model tokens	critic findings

0	0	10/10	Passed
provider retries	output repairs	accepted dispositions	EDP validation

LIVE WORKFLOW STAGE LATENCY

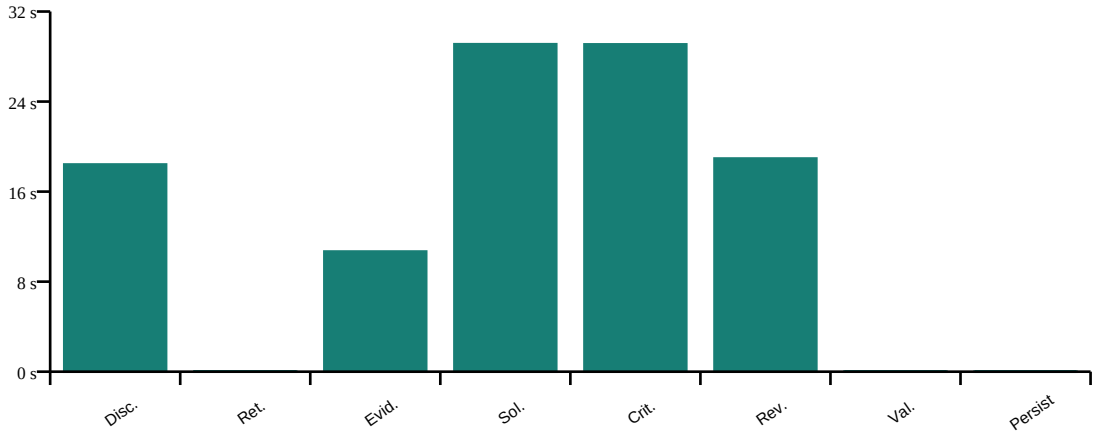


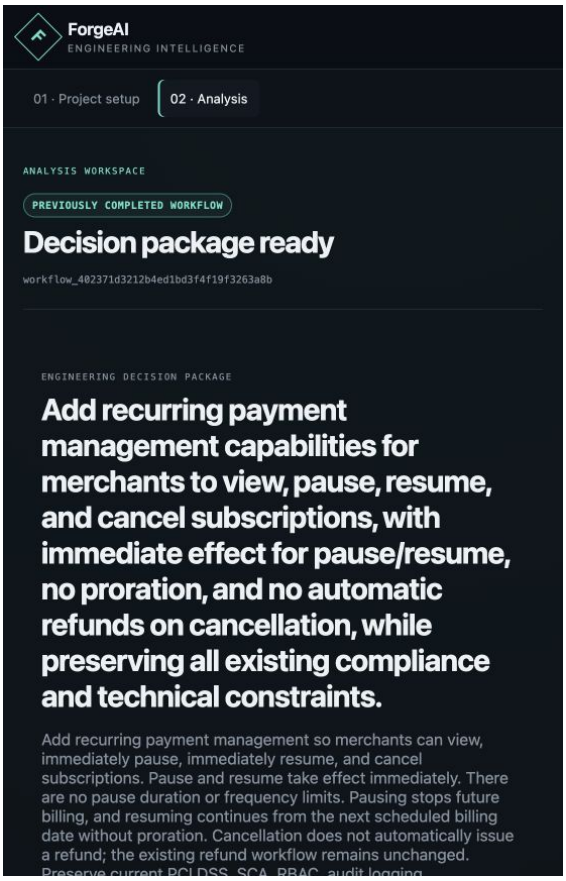
Figure 4. Persisted stage timings for workflow_402371d3212b4ed1bd3f4f19f3263a8b. Model stages dominate latency; deterministic validation and persistence are negligible.

All five LLM-backed stages - Discovery, Evidence, Solution, Critique and Revision - used AMD Radeon Cloud's served radeon-deepseek/DeepSeek-V4-Flash model.

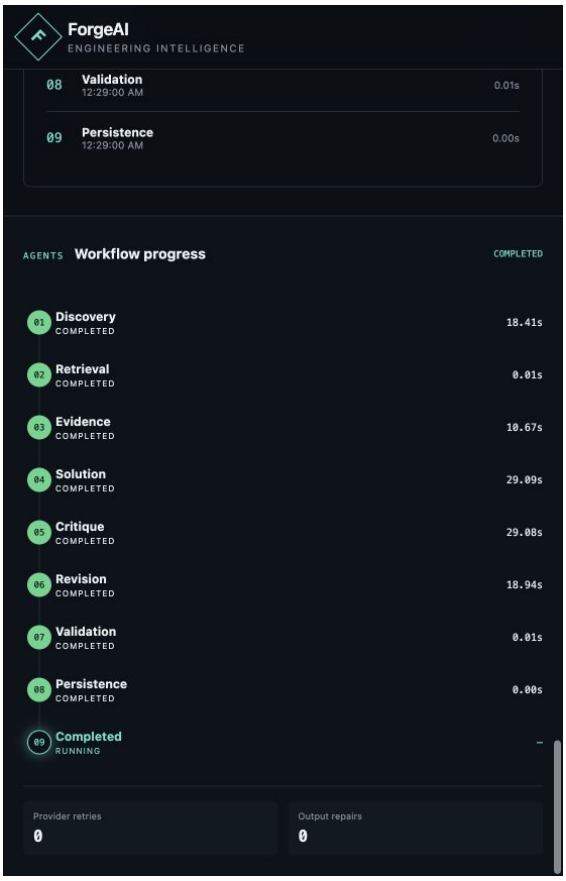
Stage	Prompt	Completion	Total	Finish	Repairs
Discovery	528	779	1,307	stop	0
Evidence	2,331	406	2,737	stop	0
Solution	2,942	1,543	4,485	stop	0
Critique	4,083	1,441	5,524	stop	0
Revision	5,621	882	6,503	stop	0
Total	15,505	5,051	20,556	-	0

12 / PRODUCT EXPERIENCE

An engineering workspace designed for user trust



Completed-package context



Persisted timeline, timings, retries and repairs

Figure 5. The product labels the record as a previously completed workflow; the replay is not represented as a live model run. Captured 06 August 2026.

The dashboard distinguishes persisted demo output from normal workflow execution. Users can inspect the decision package, stage events, evidence, architecture, delivery plan, critique, revision and provenance, then export JSON or Markdown. Normal workflow execution remains separate and unchanged.

Trust-oriented interaction

- Explicit loading, clarification, running, partial, failed and completed states.
- Ordered stage events, timings, provider retries and output repairs are visible.
- Safe failures report stable error classes without raw model output or secrets.
- Semantic HTML, keyboard focus, reduced-motion behavior and accessibility tests are implemented.

13 / DECISION ARTIFACT

A reviewable Engineering Decision Package

The completed package for recurring-payment management preserved requirements for immediate pause/resume, no proration, no automatic refund, PCI DSS, SCA, RBAC, audit logging, idempotency, webhook behavior, REST boundaries, database naming and backward compatibility.

EDP contents

- Problem definition, actors, assumptions, unknowns, user stories and acceptance criteria.
- Exact cited evidence with source provenance.
- Alternatives, recommendation, trade-offs, risks and implementation guidance.
- Claims labeled EVIDENCE, INFERENCE or UNKNOWN.
- Independent critique findings and Revision dispositions.
- Machine-readable provenance, model metadata and stage measurements.
- Validated JSON and human-readable Markdown exports.

Ten critique findings across seven review areas

Finding area	Concern tested
Idempotency	REST idempotency claims and concurrent state transitions
Reliability	Stable webhook-event identification
Backward compatibility	Webhook versioning and additive API evolution
Authorization	Merchant roles and organization-level scope
Testing	Undefined subscription states
Privacy	Audit identifiers and redaction
Performance	Missing scalability and latency requirements

The ten findings spanned the seven review areas above. Every finding received exactly one explicit disposition, and all ten were accepted and addressed in the verified run.

14 / ENGINEERING QUALITY

Production discipline inside an MVP

156	16	93%	0
backend tests passed	frontend tests passed	backend coverage	live output repairs in verified workflow

Counts were rerun and verified at commit 261a01568508a478e875d8bca47bd89be55b82f0 on 06 August 2026. Packaging was not rerun and is not claimed as freshly verified.

Verification scope

- Unit tests for domain rules, schemas, ranking, reconstruction and validation.
- Integration tests for workflows, SQLite, clarification, retries, cancellation and persistence.
- API tests for health, validation, idempotency, events, resume, EDP and CLI equivalence.
- Frontend lint, strict TypeScript checks, behavioral tests, accessibility assertions and production build.
- Mock-provider end-to-end completion without network or credentials.
- Versioned ten-case evaluation corpus for grounding, citation validity, constraint coverage, critique quality and schema validity.

Security and privacy

- Secrets enter through ignored environment configuration and are never logged or committed.
- Retrieved documents and model output are treated as untrusted input.
- No product agent can execute tools, mutate repositories or authorize workflow transitions.
- The local corpus and SQLite database remain local; external model transmission is explicit and configurable.
- API errors exclude stack traces, prompts, document content, SQL, paths and credentials.
- The backend binds conservatively to loopback; authentication is required before any non-local deployment.

15 / LOCAL RADEON / ROCM DEPLOYMENT PLAN

From the verified shared API to dedicated Radeon inference

This is a concrete planned substitution path, not a claim of completed local ROCm execution.

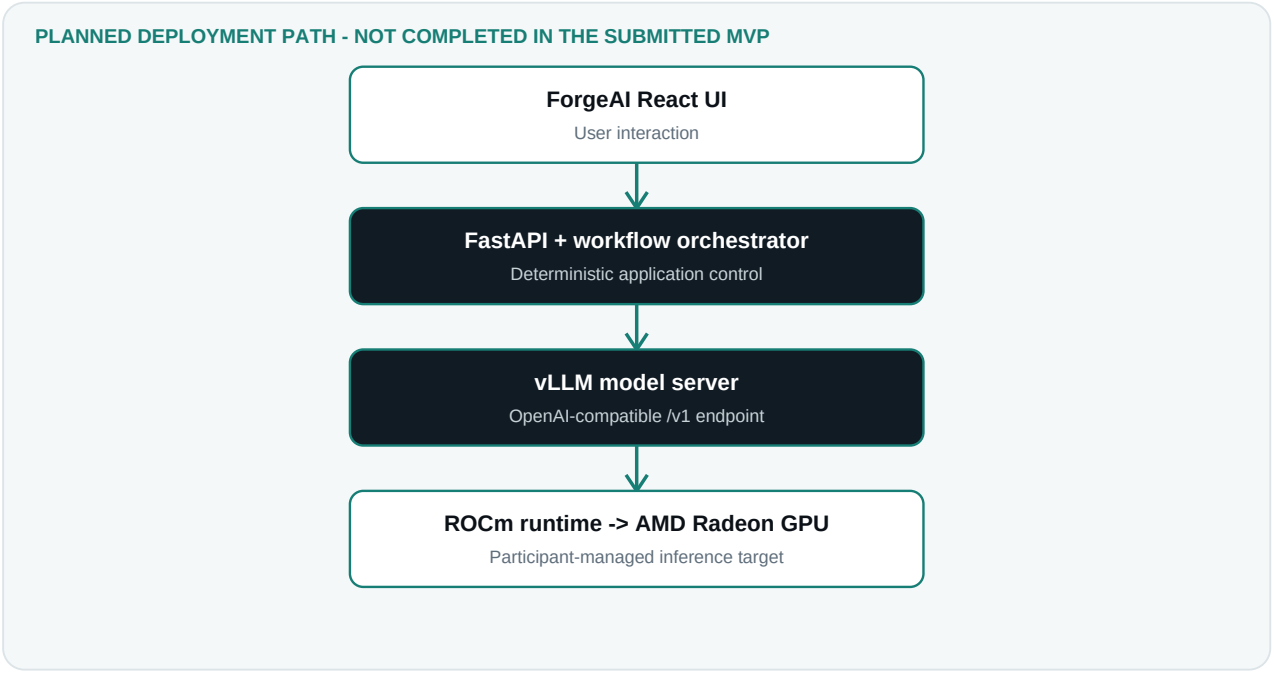


Figure 6. Planned deployment topology. The application continues to use its provider-neutral OpenAI-compatible HTTP boundary.

Numbered deployment plan

- 1. Launch a Radeon Cloud dedicated vLLM instance.
- 2. Serve a compatible open-source model through the /v1 endpoint.
- 3. Configure LLM_BASE_URL, LLM_MODEL and the deployment key.
- 4. Run ForgeAI without changing agents or orchestration.
- 5. Measure time to first token, output tokens per second, total workflow latency, GPU utilization and VRAM.
- 6. Compare baseline and quantized model configurations.

Planned deployment path; not completed in the submitted MVP.

16 / REPRODUCIBILITY

Build, run and verify from the repository checkout

Local setup

Step	Command
1. Create Python environment	python3 -m venv .venv
2. Install backend dependencies	.venv/bin/python -m pip install -r requirements.lock; .venv/bin/python -m pip install --no-deps -e .
3. Install frontend dependencies	npm --prefix frontend install
4. Initialize database	forgeai init-db
5. Index demo/mercurypay	forgeai index demo/mercurypay
6. Start FastAPI	uvicorn forgeai.api.app:create_app --factory --host 127.0.0.1 --port 8000
7. Start React	npm run dev

Live-provider environment names

Environment variable	Purpose
FORGEAI_PROVIDER_MODE	Select mock or OpenAI-compatible mode
LLM_BASE_URL	OpenAI-compatible endpoint
LLM_API_KEY	Provider credential; never committed or logged
LLM_MODEL	Explicit model selection
LLM_RESPONSE_FORMAT	Structured-output mode
LLM_TIMEOUT_SECONDS	Bounded request timeout
LLM_MAX_RETRIES	Transient provider retry limit

Verification commands

Backend: ruff format/check, mypy and pytest with coverage. Frontend: lint, typecheck, tests and production build. Contract: regenerate OpenAPI 3.1.0. Repository: git diff --check and git status.

The evaluation/v1 corpus and demo/mercurypay project are source-checkout assets. Evaluation commands that require them must run from the repository checkout.

17 / LIMITATIONS AND ROADMAP

Implemented now, extension-ready later

Area	Current state	Next evolution
Retrieval	Deterministic SQLite FTS5	Hybrid lexical/vector retrieval behind existing search port
Deployment	Local single backend and SQLite	Authenticated deployment, worker execution and PostgreSQL when justified
AMD	Shared DeepSeek API validated	Dedicated Radeon vLLM/ROCm deployment and benchmarks
Integrations	Local filesystem corpus	GitHub, Jira and enterprise document sources
Progress	HTTP polling	SSE or WebSockets if product evidence requires it
Memory	Persisted workflow context	Versioned long-term project memory
Planning	Fixed bounded pipeline	Constrained autonomous planning without weakening human control

Known limitations

- No authentication or authorization.
- No browser file upload or project-list endpoint.
- No GitHub or Jira integration.
- No embeddings or vector database.
- No external worker queue, WebSockets or SSE.
- SQLite and in-process concurrency limits remain.
- No dedicated Radeon / ROCm / vLLM deployment evidence or GPU benchmark data.
- No general long-term agent memory.
- No public hosted application.

18 / VALIDATION SCOPE

Validation Scope and Claims Boundary

Verified in the submitted MVP	Not verified or claimed
Public repository and source code	Participant-managed dedicated Radeon deployment
Complete AMD-hosted DeepSeek workflow	Local ROCm model serving
Persisted EDP JSON and Markdown	GPU utilization or VRAM measurement
Model and token telemetry	Quantization
Test suite and 93% backend coverage	Kernel-level tuning
Product UI and completed-workflow viewer	Dedicated-vLLM benchmarking
Provider-neutral architecture	Local inference execution in the submitted MVP

All implementation and live-run claims are traceable to the pinned repository or persisted workflow. Planned capabilities are labeled and do not substitute for missing hardware measurements.

19 / PARTICIPANT CONTRIBUTION

Individual authorship and AI-assisted implementation

Abhishek Herbert Samuel participated individually and defined ForgeAI's problem, product scope, architecture decisions, acceptance criteria, AMD integration strategy, validation approach and submission. Codex was used as an AI-assisted engineering implementation tool under participant direction. The AMD DeepSeek model generated the five bounded stage drafts in the verified live workflow, while ForgeAI's deterministic software owned retrieval, provenance, validation, reconstruction, orchestration, persistence and final EDP assembly.

Responsibility boundary

Contributor or system	Role
Abhishek Herbert Samuel	Solo participant, product owner, architectural decision-maker and submission author
Codex	AI-assisted engineering tool used under participant direction; not a team member
AMD DeepSeek model	Generated five bounded model drafts in the verified workflow; not a team member
ForgeAI deterministic software	Owned retrieval, trusted metadata, validation, state, persistence and EDP assembly

20 / CONCLUSION

Controlled agentic engineering on AMD

ForgeAI demonstrates that useful agentic engineering does not require surrendering control to an autonomous model. By combining local retrieval, bounded reasoning roles, trusted provenance, human clarification, independent critique and deterministic validation, ForgeAI converts ambiguous requests into inspectable Engineering Decision Packages. The submitted MVP completed a full five-agent workflow through AMD Radeon Cloud and is architecturally ready for future dedicated Radeon/ROCm inference.

5	20,556	10/10	Passed
bounded agent roles	verified model tokens	critic findings dispositioned	final EDP validation

Submission claim

Implemented: private engineering workflow, local lexical retrieval, deterministic evidence reconstruction, validated EDP generation, CLI, REST API and React UI. Verified: one complete DeepSeek-V4-Flash workflow through AMD Radeon Cloud. Planned: participant-managed dedicated Radeon, ROCm and vLLM inference with GPU performance measurement.

21 / SOURCE BASIS

Auditable references

Reference	Repository path or URL
Engineering constitution	AGENTS.md
Project overview and runbook	README.md
Architecture and roadmap	docs/implementation-plan.md
Agent contracts	docs/agent-contracts.md
Workflow state machine	docs/workflow-orchestration.md
Engineering Decision Package	docs/engineering-decision-package.md
Provider abstraction	docs/model-provider.md
REST API	docs/api.md
Frontend architecture	docs/frontend.md
Evaluation framework	docs/evaluation.md
OpenAPI contract	contracts/openapi.json
Evaluation dataset	evaluation/v1/dataset.json
Live workflow JSON	evaluation-results/live-discovery-budget-workflow_402371d3212b4ed1bd3f4f19f3263a8b.json
Live EDP Markdown	evaluation-results/live-discovery-budget-workflow_402371d3212b4ed1bd3f4f19f3263a8b.md
Architecture decisions	docs/adr/0001 through docs/adr/0008
Official public hackathon and Track 2 rules	https://huggingface.co/blog/LeRobot-worldwide-hackathon/amd-ai-devmaster
Official event registration	https://luma.com/amd-4dhi
ForgeAI GitHub repository	https://github.com/AbhishekHerbertSamuel/forgeai
Exact verified commit	https://github.com/AbhishekHerbertSamuel/forgeai/commit/261a01568508a478e875d8bca47bd89be55b82f0

Document control

Field	Value
Specification date	06 August 2026
Repository commit	261a01568508a478e875d8bca47bd89be55b82f0
Application source	Unchanged at pinned commit; PDF outputs intentionally uncommitted
Product version	0.1.0
PDF filename	ForgeAI-Project-Specification-AMD-Track-2.pdf
PDF specification version	2.0 - Track 2 submission revision